

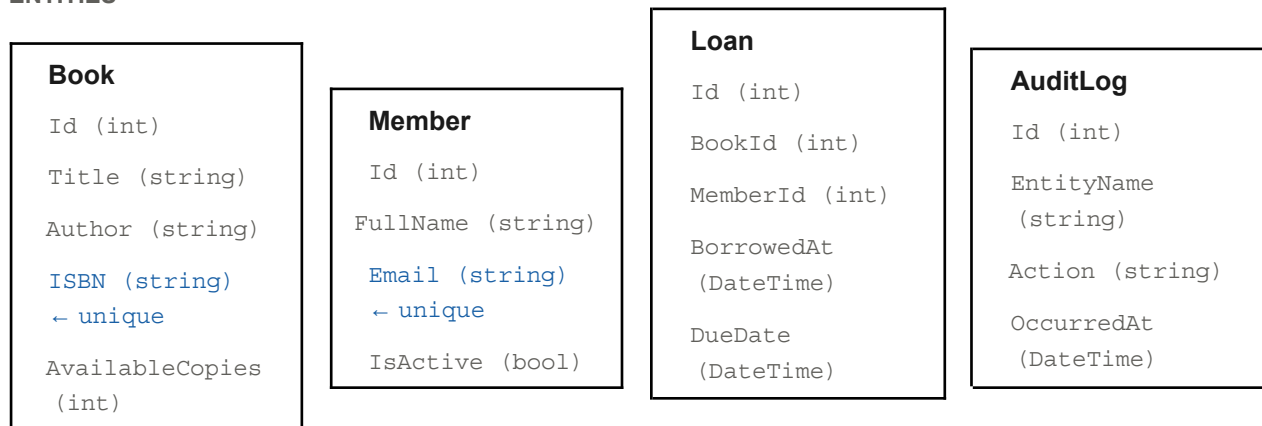
Homework: Library Management

API Entity Framework Core — ASP.NET Core Web API

WHAT YOU ARE BUILDING

You are building a small **Library Management API** using ASP.NET Core Web API and Entity Framework Core. The API will let users add books, register members, borrow a book, and view all active loans. You will be given the entities below — your job is to wire everything up using EF Core correctly.

ENTITIES



BEFORE WRITING ANY ENDPOINT — BUILD THE AUDIT INTERCEPTOR

You must create a **SaveChangesInterceptor** that automatically logs every database write to the [AuditLog](#) table. This interceptor must be working before you test any endpoint, because it fires silently in the background on every save — you should never call it manually.

- 1 Create a class called [AuditInterceptor](#) that inherits from [SaveChangesInterceptor](#).
- 2 Override the [SavingChangesAsync](#) method. Inside it, loop through `context.ChangeTracker.Entries()` and find every entry whose state is `Added`, `Modified`, or `Deleted`.
- 3 For each such entry, create a new [AuditLog](#) row — set `EntityName` to the entity type name, `Action` to the state (e.g. "Added"), and `OccurredAt` to `DateTime.UtcNow`.
- 4 Add the new rows using `context.Set<AuditLog>().AddRange(...)` — do this **before** calling `base.SavingChangesAsync()`.
- 5 Register the interceptor in your [LibraryDbContext](#) by overriding [OnConfiguring](#) and

calling `optionsBuilder.AddInterceptors(new AuditInterceptor())`.

ENDPOINTS TO IMPLEMENT

POST `/api/books`

Add a new book to the library. Save it to the database and return the created book with its generated Id.

EF CORE REQUIREMENTS

- Configure `Book` using Fluent API — set max length 150 on Title, make ISBN required and unique.
- Use `DbContext.SaveChangesAsync()` to persist — the interceptor will fire automatically.
- Generate and apply a migration so the Books table exists in the database.

POST `/api/members`

Register a new library member. Set `IsActive = true` automatically on creation.

EF CORE REQUIREMENTS

- Configure `Member` using Fluent API — Email must be required, unique, and max 200 chars.
- Use `HasDefaultValue(true)` on `IsActive` in Fluent API configuration.

POST `/api/loans`

Borrow a book. Accepts a `BookId` and `MemberId`. Creates a loan record and decreases `Book.AvailableCopies` by 1. Return 400 if no copies are available.

EF CORE REQUIREMENTS

- Load the book using `DbContext`, modify `AvailableCopies`, then call `SaveChangesAsync()` — EF Core's change tracker detects the modification automatically. Do not write a manual UPDATE query.
- After saving, verify that your `AuditInterceptor` created two rows in `AuditLog` — one for the new Loan (Added) and one for the updated Book (Modified).

GET `/api/loans`

Return all loan records including the book title and member name. Only loans belonging to active members should appear.

EF CORE REQUIREMENTS

- Use `.Include()` to eagerly load the related `Book` and `Member` for each loan.
- Add a global query filter on `Member` in Fluent API: `.HasQueryFilter(m => m.IsActive)`. This means you do not need a `.Where()` in this endpoint — inactive members are excluded automatically.